
High Performance Computing

Lecture 1

Prerequisites

- (Intro to Computer Architecture/Systems course) is a strict prerequisite for this course.
- Students are expected to be strong C/C++ programmers as there will be exposure to a variety of "C-like" parallel programming languages in this course.

Textbooks

- Introduction to Parallel Computing by Grama, Gupta, Karypis and Kumar, second edition, Pearson
- Parallel Programming in C with MPI and OpenMP by Michael J Quinn, TATA McGrawHill
- Using MPI, Portable Parallel Programming with the Message-Passing Interface by Gropp, Lusk and Skjellum, third edition, MIT Press
- Culler & Singh - *Parallel Computer Architecture: A Hardware/Software Approach* – Morgan Kaufmann
- Hennessy & Patterson - *Computer Architecture: A Quantitative Approach* – Morgan Kaufmann – 5th edition

Course Assessments

- Lectures
 - Higher level concepts
 - Provide in-depth understanding of some aspect of Systems
- Quizzes, Attendance (30%)
- Two midterms exams (30%)
- Final exam (40%)

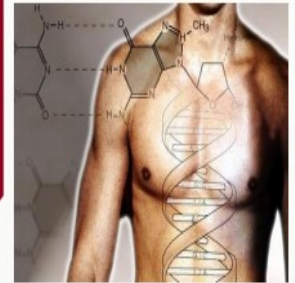
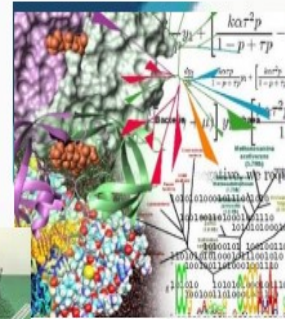
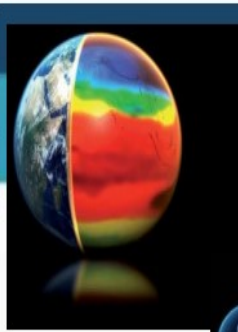
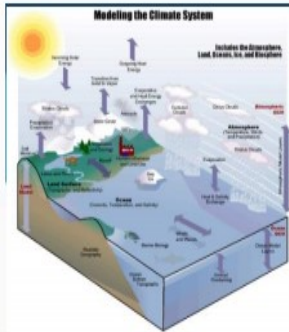
Outline

- Introduction
- Parallel Machines
- Multicores
- Parallel Programming Models
- Job Management Systems

What is HPC?

- High Performance Computing (HPC) refers to the technology of aggregating computing power in a way that delivers much higher power than traditional computers and servers.
- HPC is the way of processing huge volumes of data at very high speeds using multiple computers and storage devices.
- HPC makes it possible to explore and find answers to some of the world's biggest problems in science, engineering, and business.

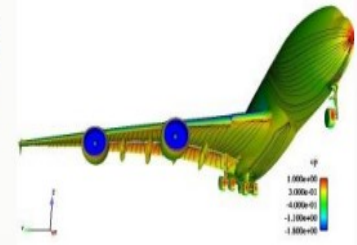
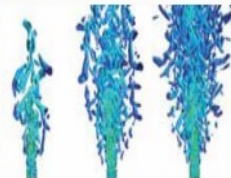
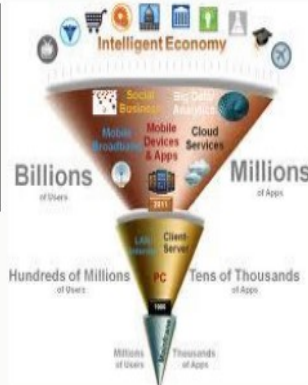
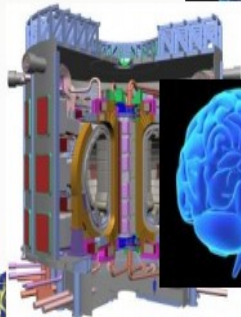
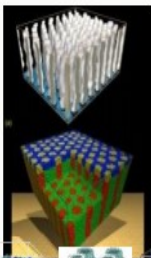
HPC: What for?



Bio/Life Sciences

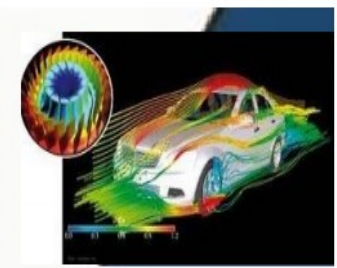
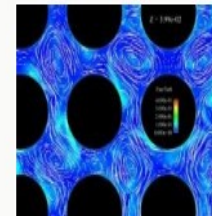
Weather, Climate & Earth Sciences

**New applications
e.g. Health, Big data**



**Fundamental sciences: Physics,
Chemistry, Material Sciences,
Astrophysics**

Industrial & Engineering



What is Parallel Computing?

- Consider your favorite computational application
 - One processor can give me results in N hours
 - Why not use N processors
 - and get the results in just one hour?

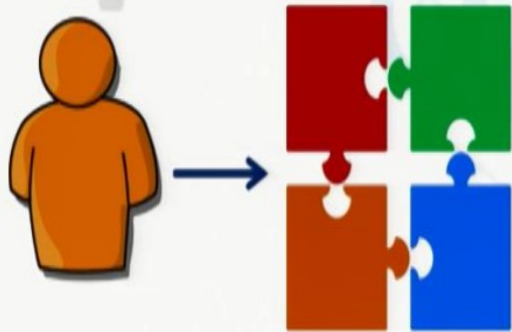
The concept is simple:

Parallelism = applying multiple processors to a single problem

Serial Computing- vs - Parallel Computing

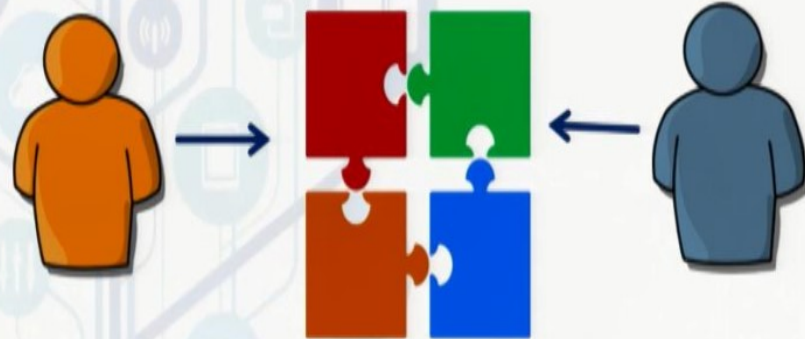
Serial Computing -vs- Parallel Computing

Serial (sequential) Computing



One person solves a 2000 puzzles
Takes 10 hours

Parallel Computing



Orange and gray guys distributes the
jobs among themselves

Solves the puzzles in 5 hour 30 mins

The Basic Idea of Parallel Computing?

Parallel Computing – the Basic Idea



**Serial/ sequential/
standalone computing**

- The load(memory) can be too heavy to carry
- Task will take astronomically long time!

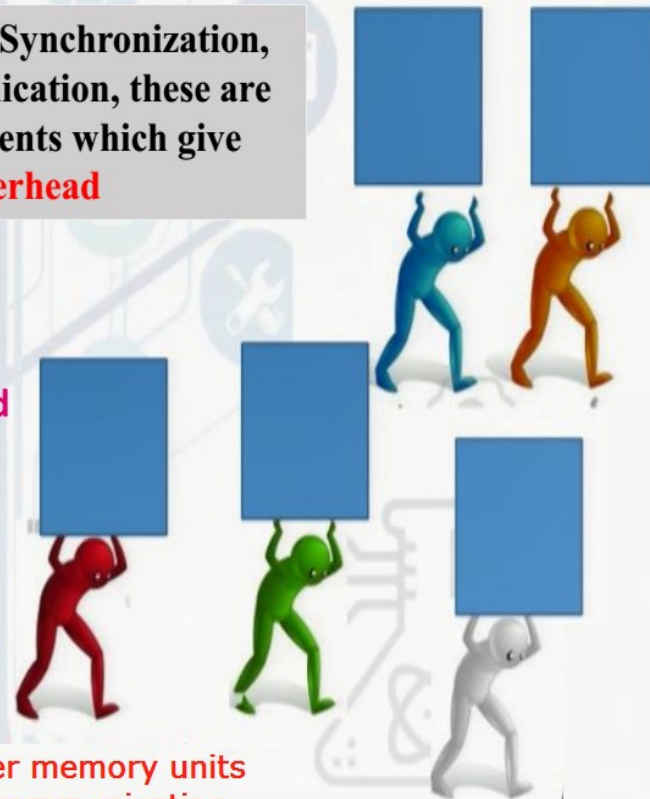
Parallel computing



**Shared
memory
(load)**

Distribution, Synchronization,
and Communication, these are
the components which give
overhead

**Distributed
Memory**

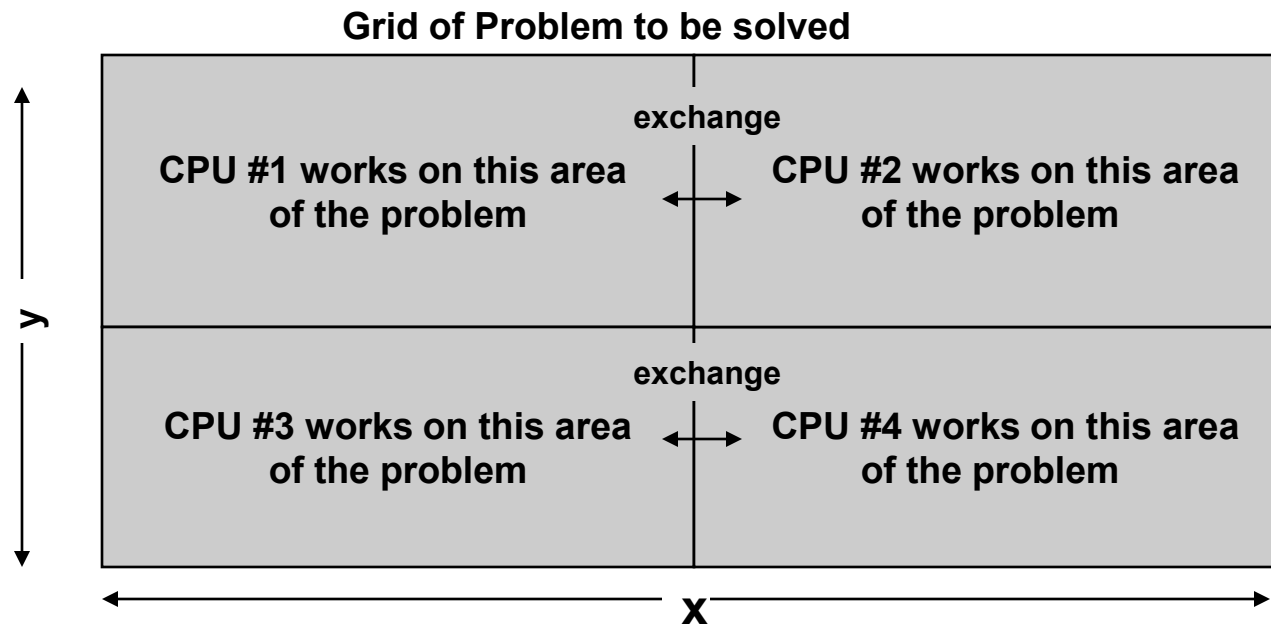


**Smaller memory units
More communication**

**Large Piece of Memory (or Load) to be used
More Synchronization, Less Communication**

Parallel computing

- Parallel computing: the use of multiple computers or processors working together on a common task.
 - Each processor works on its section of the problem
 - Processors are allowed to exchange information with other processors



Why Do Parallel Computing?

- Limits of single CPU computing
 - Available memory
 - Performance
- Parallel computing allows:
 - Solve problems that don't fit on a single CPU's memory space
 - Solve problems that can't be solved in a reasonable time

Types of Parallelism : Two Extremes

- **Data parallel**
 - Each processor performs the same task on different data
- **Task parallel**
 - Each processor performs a different task
- **Most applications fall somewhere on the continuum between these two extremes**

Basics of Data Parallel Programming

One code will run on 2 CPUs

Program has array of data to be operated on by 2 CPU so array is split into two parts.

program:

```
...
if CPU=a then
    low_limit=1
    upper_limit=50
elseif CPU=b then
    low_limit=51
    upper_limit=100
end if
do I = low_limit,
upper_limit
    work on A(I)
end do
...
end program
```

CPU A

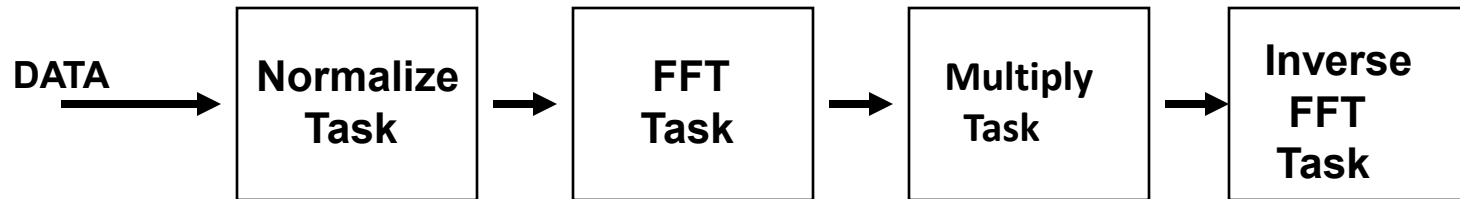
```
program:
...
low_limit=1
upper_limit=50
do I= low_limit,
upper_limit
    work on A(I)
end do
...
end program
```

CPU B

```
program:
...
low_limit=51
upper_limit=100
do I= low_limit,
upper_limit
    work on A(I)
end do
...
end program
```

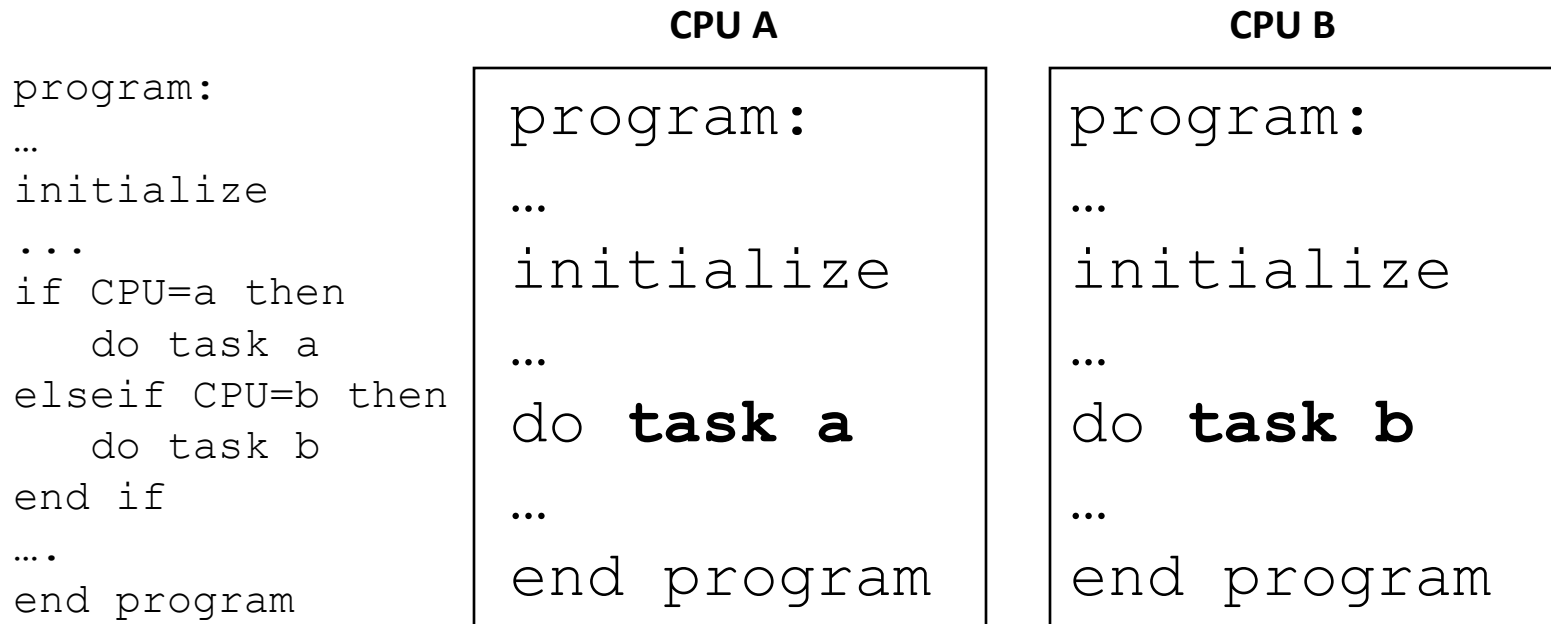
Typical Task Parallel Application

- Example: Signal Processing
- Use one processor for each task
- Can use more processors if one is overloaded



Basics of Task Parallel Programming

- One code will run on 2 CPUs
- Program has 2 tasks (a and b) to be done by 2 CPUs

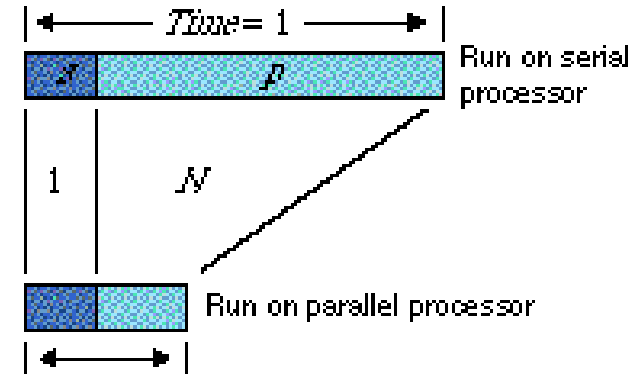


Limits of Parallel Computing

- Theoretical Upper Limits
 - Amdahl's Law
- Practical Limits
 - Load balancing
 - Non-computational sections
- Other Considerations
 - time to re-write code

Theoretical Upper Limits to Performance

- All parallel programs contain:
 - Serial sections
 - Parallel sections



- Serial sections – when work is duplicated or no useful work done (waiting for others) - limit the parallel effectiveness
 - Lot of serial computation gives bad speedup
 - No serial work “allows” perfect speedup

Amdahl's Law

- **Amdahl's Law places a strict limit on the speedup that can be realized by using multiple processors.**
 - **Effect of multiple processors on run time**

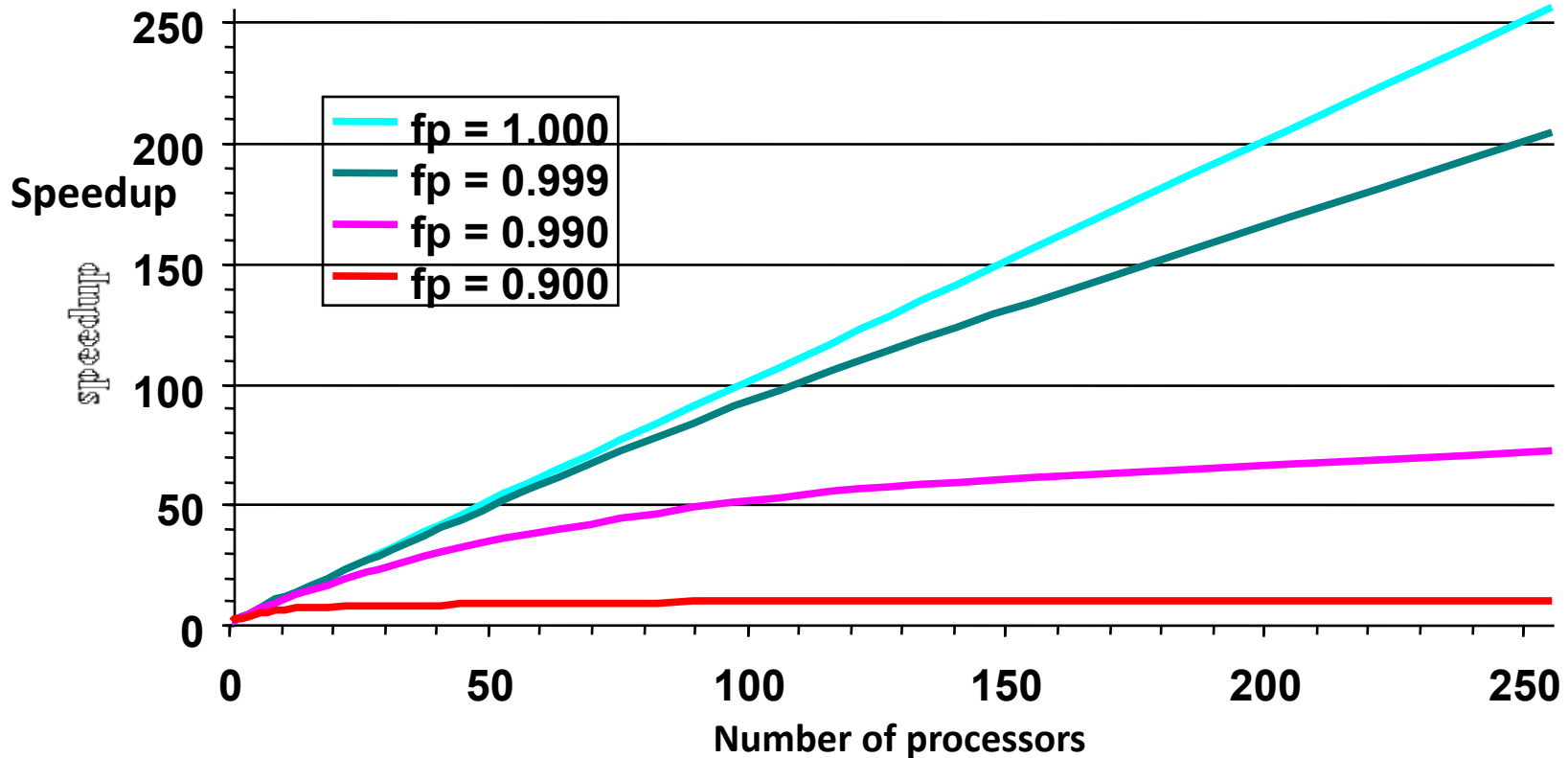
$$t_n = \left(f_p / N + f_s \right) t_1$$

- **Effect of multiple processors on speed up ($S = t_1/t_n$)**

- **Where**
$$S = \frac{1}{f_s + f_p / N}$$
 - **f_s = serial fraction of code**
 - **f_p = parallel fraction of code**
 - **N = number of processors**
 - **t_n = time to run on N processors**

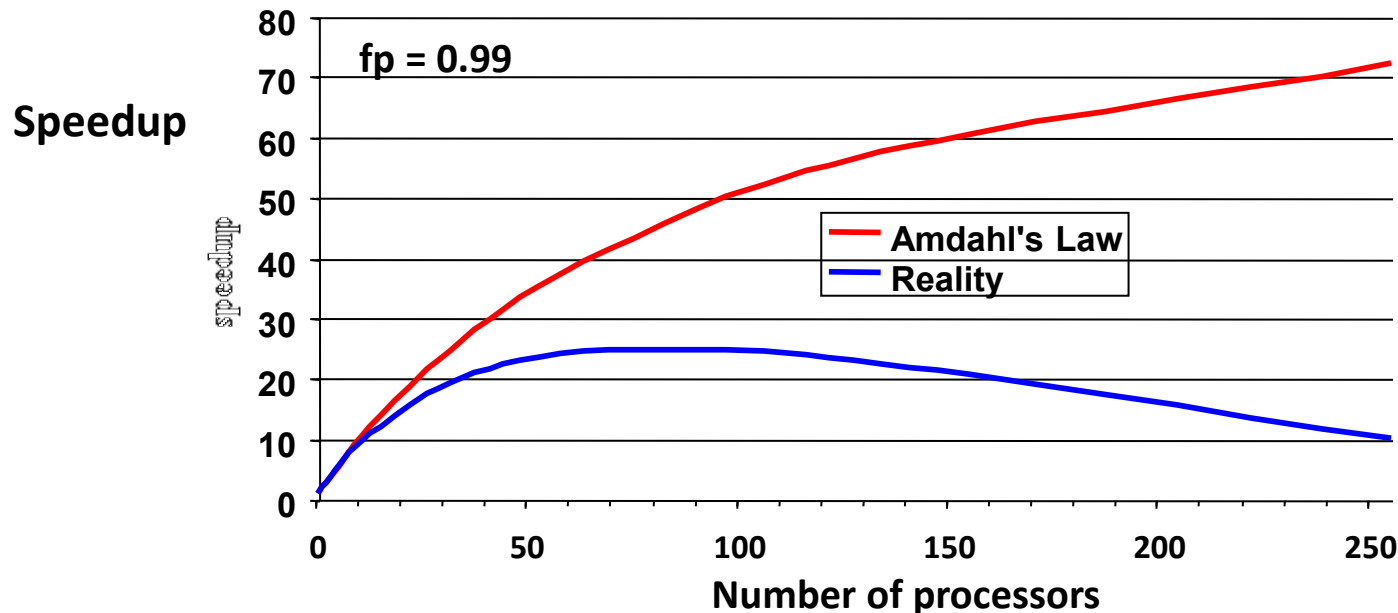
Illustration of Amdahl's Law

It takes only a small fraction of serial content in a code to degrade the parallel performance.



Amdahl's Law vs. Reality

- Amdahl's Law provides a theoretical upper limit on parallel speedup assuming that there are no other costs for speedup
- In reality, Amdahl's Law is limited by many things:
 - Communications
 - I/O
 - Load balancing (waiting)
 - Scheduling (shared processors or memory)



Outline

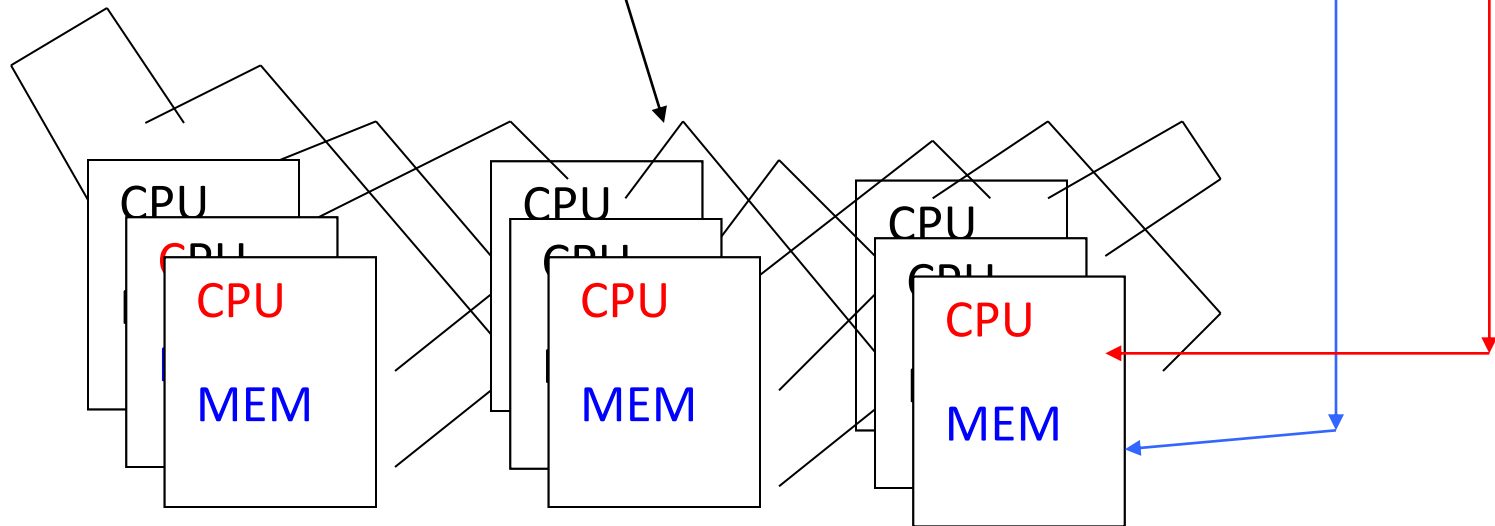
- Introduction
- **Parallel Machines**
- Multicores
- Parallel Programming Models
- Job Management Systems

Simplistic Architecture

- **Processors**

- **Memory**

- Interconnect Network



Flynn's Taxonomy

- Many attempts have been made to come up with a way to categorize computer architectures.
- *Flynn's Taxonomy* has been the most enduring of these, despite having some limitations.
- Flynn's Taxonomy takes into consideration the number of processors and the number of data paths incorporated into an architecture.
- A machine can have one or many processors that operate on one or many data streams.

Flynn's Taxonomy

- The four combinations of multiple processors and multiple data paths are described by Flynn as:
 - **SISD**: Single instruction stream, single data stream. These are classic uniprocessor systems.
 - **SIMD**: Single instruction stream, multiple data streams. Execute the same instruction on multiple data values, as in vector processors.
 - **MIMD**: Multiple instruction streams, multiple data streams. These are today's parallel architectures.
 - **MISD**: Multiple instruction streams, single data stream.

Single Instruction Single Data

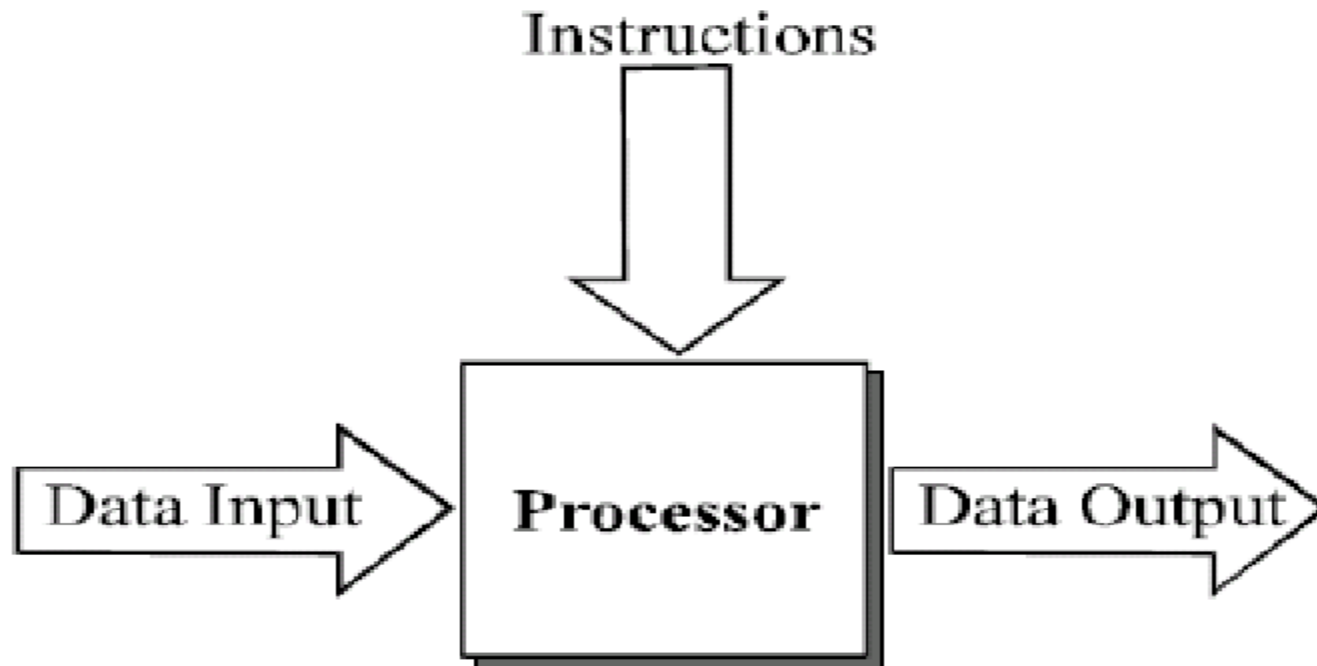


Figure 1.2: SISD Architecture

Single Instruction Multiple Data

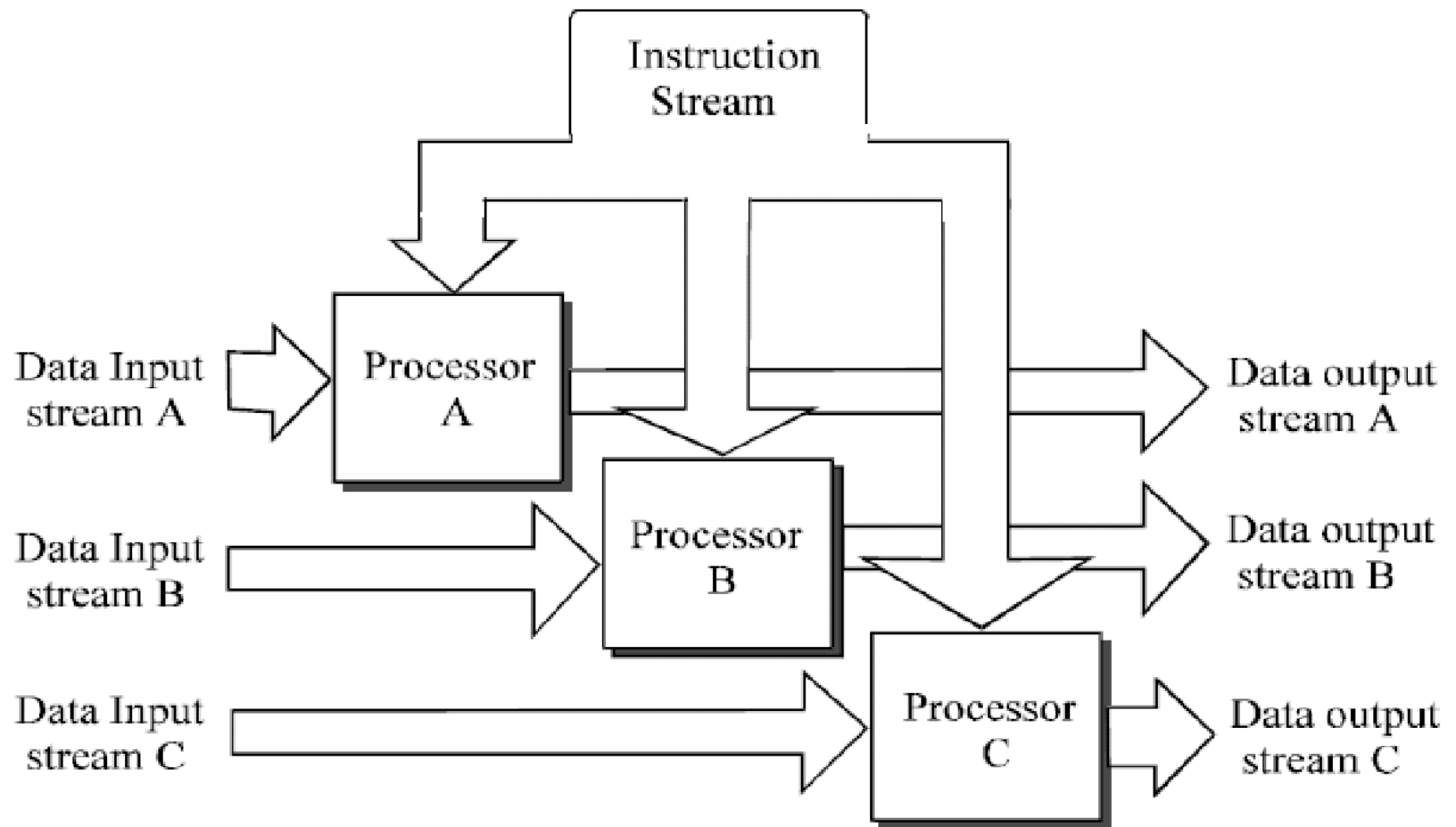


Figure 1.3: SIMD Architecture

Multiple Instruction Single Data

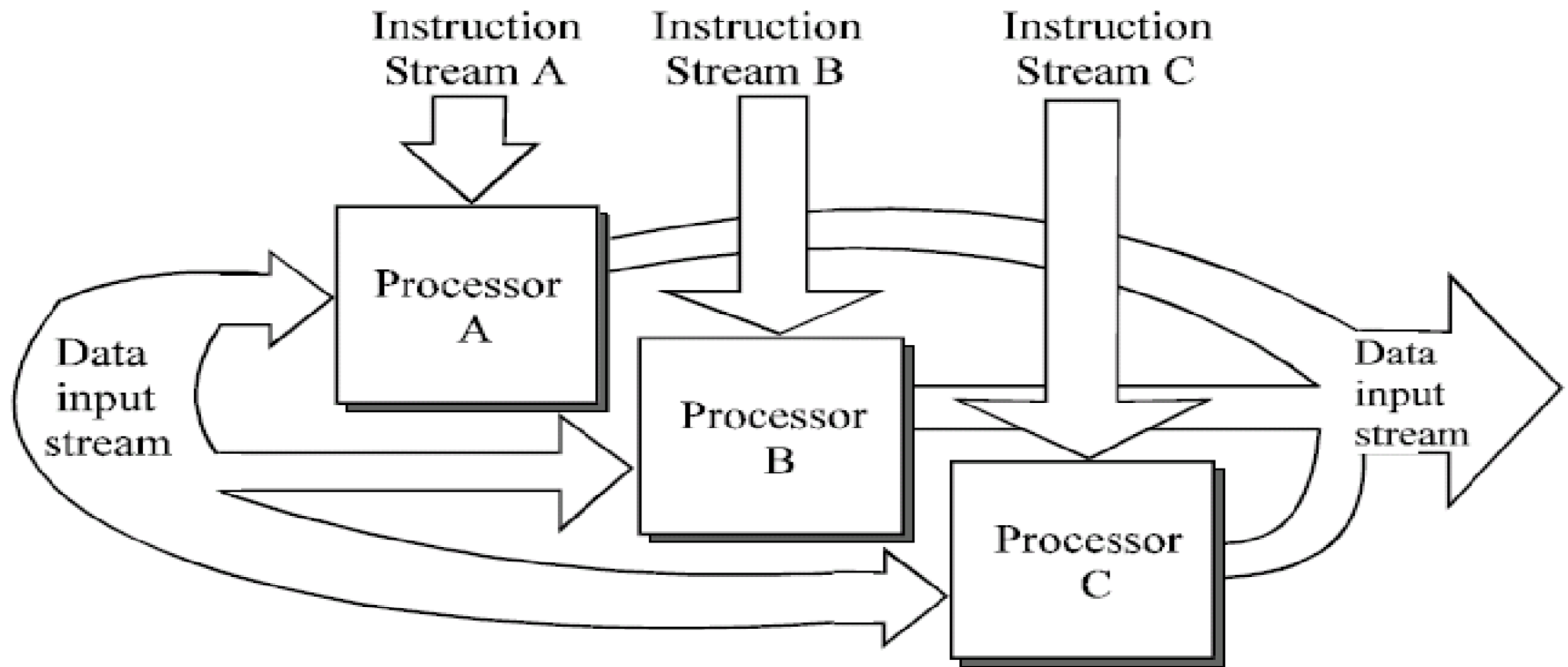


Figure 1.4: MISD architecture

Multiple Instruction Single Data

- MISD computing system is a multiprocessor machine capable of Executing different instructions on different Processing Elements (PEs), but all of them operating on the same data-set. For instance, statements such as: Multicores
- MISD Perform different operations on data set. Machines built using MISD model, are not useful in most of the applications; a few machines are built, but none of them are available commercially.
- MISD become more of an intellectual exercise than a practical configuration.

Multiple Instruction Multiple Data

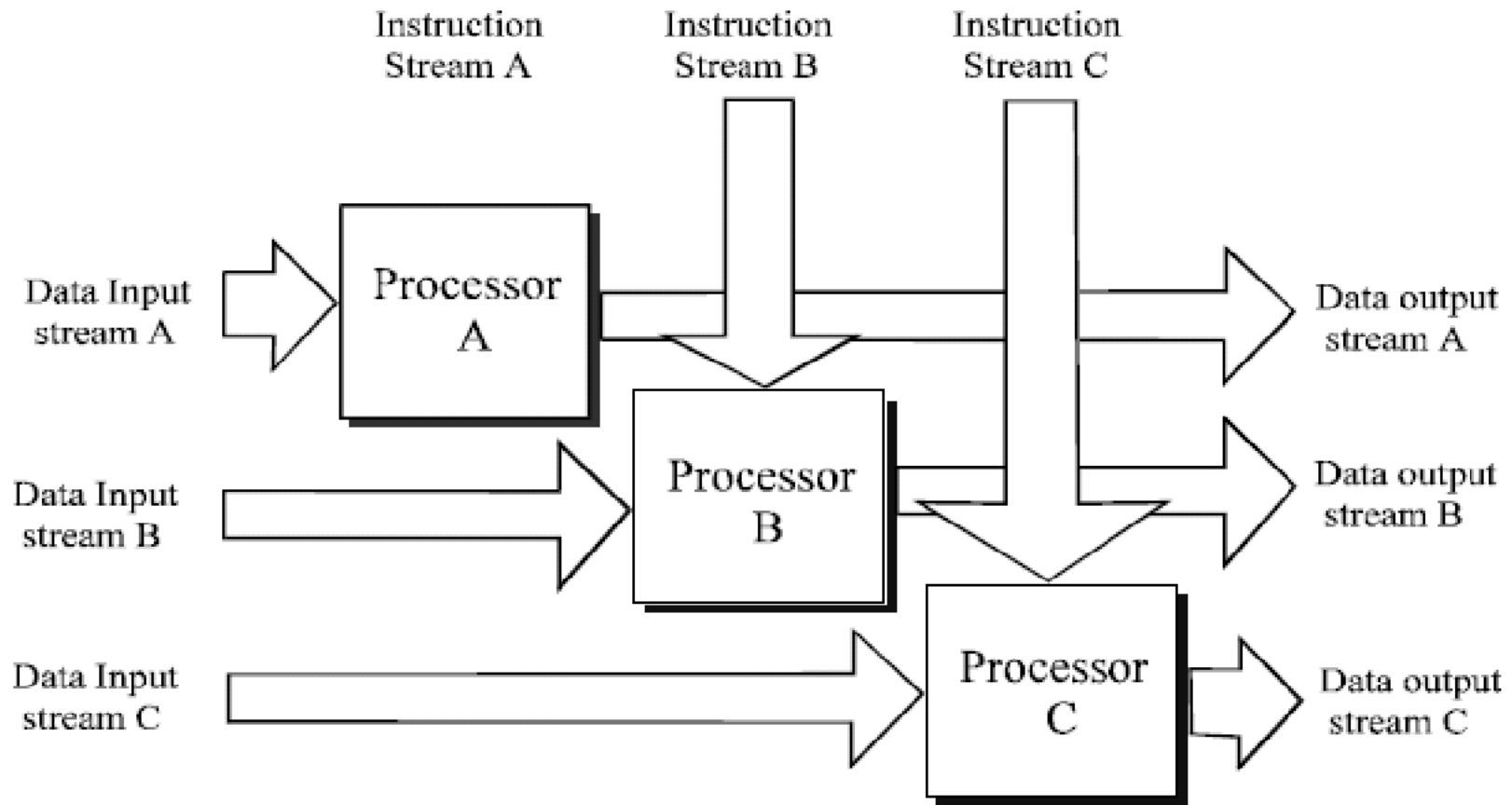


Figure 1.5: MIMD architecture

Multiple Instructions Multiple Data

- MIMD computing system is a multiprocessor machine capable of executing multiple instructions or multiple data sets
- Each PEs in MIMD model have separate instruction and data stream
- MIMD are suited for any kind of applications.
- Processing Elements (PEs) in MIMD machines work asynchronously

Flynn's Taxonomy

- Symmetric multiprocessors (SMP) and massively parallel processors (MPP) are MIMD architectures that differ in how they use memory.
- SMP systems share the same memory and MPP do not.
- An easy way to distinguish SMP from MPP is:
 - MPP $\Rightarrow$ many processors + distributed memory + communication via network
 - SMP $\Rightarrow$ fewer processors + shared memory + communication via memory

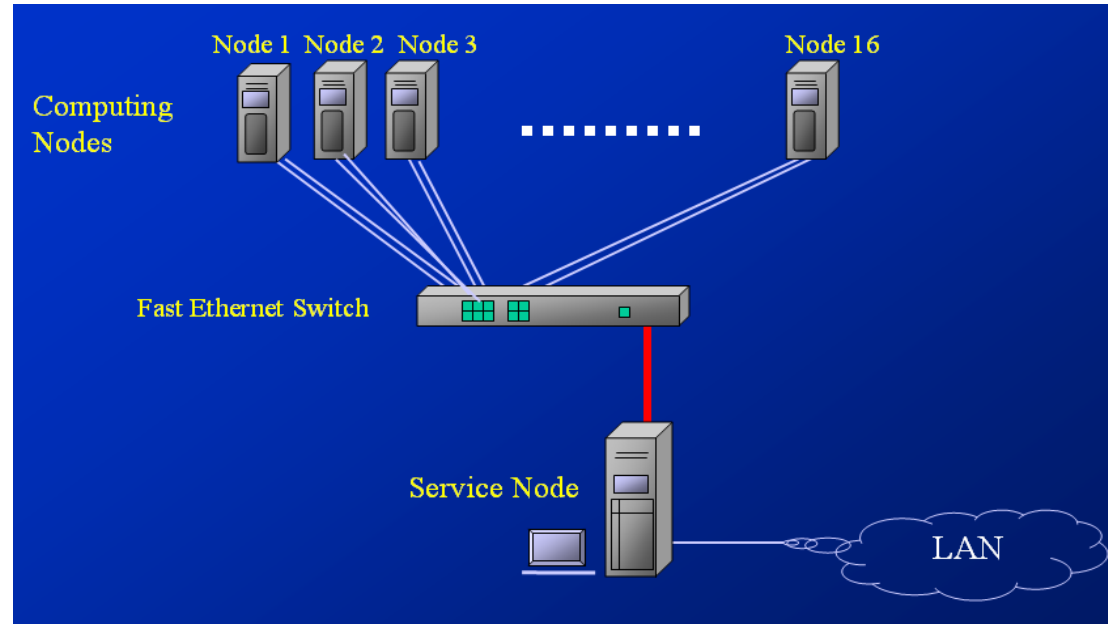
The Grid

- An infrastructure that enables the integrated, collaborative use of high-end computers, networks, and databases owned and managed by multiple organizations



Clusters

A cluster is a type of parallel or distributed processing system, which consists of a collection of interconnected stand-alone or complete computers. These computers cooperatively work together as a single, integrated computing resource.



■ A typical Cluster :

- one computer called *service node* or *front node* (used to interact with users and manage the cluster)
- a group of computers called *computing node* (no keyboard, mouse, floppy, video...)
- communications between nodes on an interconnect network platform (Ethernet, Myrinet....)
- The computing nodes are separated from outside networks and dedicate to computing. The service node is a gateway machine of the whole system.

Any Questions



Thank You